In **C# Object-Oriented Programming**, a **class** contains different **members** that define its data and behavior. Among the most fundamental members are **variables (fields)** that store data inside the class.

Today we will focus on four important types of variables:

1. **Instance Variables**
2. **Static Variables**
3. **Const Variables**
4. **Readonly Variables**

These are also called **fields** in C#.

---

1. Instance Variables (Instance Fields)

Definition

An **instance variable** is a variable that belongs to an **object (instance) of a class**.

Every time you create a new object, **a separate copy of the instance variable is created**.

Key Characteristics

- Declared inside a class but **outside methods**
- **Each object has its own copy**
- Can be accessed using the **object name**
- Stored in **heap memory** (inside the object)

Syntax

```
class Student
{
   public int studentId;
   public string studentName;
}
```

Example

```
class Student
{
    public int id;
    public string name;
}

class Program
{
    static void Main()
    {
        Student s1 = new Student();
        Student s2 = new Student();

        s1.id = 1;
        s1.name = "Rahul";

        s2.id = 2;
        s2.name = "Amit";

        Console.WriteLine(s1.name);
        Console.WriteLine(s2.name);
    }
}
```

Output

Rahul

Amit

Explanation

Two objects are created:

Student s1

Student s2

Memory structure:

Object s1

  id = 1

  name = Rahul

Object s2

  id = 2

  name = Amit

Each object has **separate memory**.

When to Use Instance Variables

Use them when the **value differs for each object**.

Example:

- Student Name
- Employee Salary
- Product Price
- Account Balance

---

2. Static Variables

Definition

A **static variable** belongs to the **class itself**, not to the object.

There is **only one copy** of the variable shared by **all objects**.

Key Characteristics

- Declared using the **static keyword**
- Shared by all objects
- Stored in **static memory**
- Accessed using **class name**

Syntax

```
class Employee
{
    public static string companyName = "Infosys";
}
```

Example

```
class Employee
{
    public int empId;
    public string name;

    public static string company = "TCS";
}

class Program
{
    static void Main()
    {
```

```
        Employee e1 = new Employee();
        Employee e2 = new Employee();

        Console.WriteLine(Employee.company);
    }
}
```

Output

TCS

Memory Representation

```
Class Employee
  static company = TCS

Object e1
  empId
  name

Object e2
  empId
  name
```

Only **one copy of company exists**.

When to Use Static Variables

Use them when the value is **common for all objects**.

Examples:

- Company Name

- College Name
- Country Name
- Application Configuration

---

3. Const Variables

Definition

A **const variable** is a **compile-time constant** whose value **cannot be changed after declaration**.

Key Characteristics

- Must be initialized **at the time of declaration**
- Value **cannot change**
- **Implicitly static**
- Known at **compile time**

Syntax

```
const datatype variableName = value;
```

Example

```
class MathConstants
{
    public const double PI = 3.14159;
}
```

Usage:

```
class Program
{
    static void Main()
    {
```

```
        Console.WriteLine(MathConstants.PI);
    }
}
```

Important Rules

Rule 1: Must assign value during declaration

Correct:

```
const int x = 10;
```

Wrong:

```
const int x;
x = 10;  // Not allowed
```

Rule 2: Cannot change value

Wrong:

```
const int x = 10;
x = 20;   // Compile time error
```

Rule 3: Cannot be modified anywhere

```
class Test
{
    const int a = 5;

    void Change()
    {
        // a = 10; ❌ Not allowed
    }
```

```
}
```

When to Use Const

Use when value **never changes**.

Examples:

- PI value
- Days in week
- Maximum size
- Mathematical constants

---

4. Readonly Variables

Definition

A **readonly variable** is a variable whose value **can be assigned only once**, but **not necessarily at declaration**.

It can be assigned:

1. At declaration
2. Inside constructor

After that it **cannot be modified**.

Syntax

```
readonly datatype variableName;
```

Example

```
class Student
{
    public readonly int id;
```

```
    public Student(int studentId)
    {
        id = studentId;
    }
}
```

Usage:

```
class Program
{
    static void Main()
    {
        Student s1 = new Student(101);
        Console.WriteLine(s1.id);
    }
}
```

Key Rule

Readonly value can be set in:

1️⃣ **Declaration**

```
readonly int x = 10;
```

2️⃣ **Constructor**

```
readonly int x;

public Test()
{
    x = 10;
```

```
}
```

But **cannot change later**.

Const vs Readonly (Very Important Interview Question)

Feature	const	readonly
Initialization	At declaration only	Declaration or constructor
Time	Compile time	Runtime
Static	Always static	Can be instance
Value change	Never	Once (during initialization)
Memory	Stored in metadata	Stored in object

Example Comparison

```
class Test
{
    public const int a = 10;
    public readonly int b;

    public Test()
    {
        b = 20;
    }
}
```

Complete Example

```
class Demo
```

```
{
    public int instanceVar = 10;

    public static int staticVar = 20;

    public const int constVar = 30;

    public readonly int readOnlyVar;

    public Demo()
    {
        readOnlyVar = 40;
    }
}

class Program
{
    static void Main()
    {
        Demo d = new Demo();

        Console.WriteLine(d.instanceVar);
        Console.WriteLine(Demo.staticVar);
        Console.WriteLine(Demo.constVar);
        Console.WriteLine(d.readOnlyVar);
    }
```

}

Quick Summary

Variable Type	Belongs To	Copies	When Assigned
Instance	Object	One per object	Runtime
Static	Class	Single copy	Runtime
Const	Class	Single copy	Compile time
Readonly	Object/Class	One per object	Runtime (constructor)

✅ **Simple way to remember**

Instance → Object specific

Static → Class shared

Const → Compile time constant

Readonly → Runtime constant